Still Needed, Hard to Transfer: Using and Reusing Contextual Skills

Anonymous Author(s)

Abstract

Language models can adapt to new tasks by following instructions provided in context. Reusing the internal states formed from these instructions could allow the resulting task capabilities to be shared across executions. However, it remains unclear how the ability to transfer these states relates to the model's continued dependence on the instructions. We study this relationship using skills, sets of instructions that describe how to perform a task. At different layers, we transfer states at the instruction positions from an execution using the correct skill into one using an unrelated skill. Separately, we block attention to the skill in the original execution. Across four models and two benchmarks, all eight comparisons show that transfer weakens at later layers while the original execution remains strongly dependent on the skill. We then augment state transfer with key/value caches, the stored information used by attention. At layer 16, this joint intervention improves accuracy over state transfer alone by 6.8 to 43.4 percentage points across all eight combinations. For Qwen3-8B/MedCalc, accuracy rises from 3.5% to 42.1%, and blocking subsequent skill reading reduces it to 7.0%. Complete-source controls further show that donor skill-position KV alone does not recover donor performance; the depth dependence of joint repair varies across models and tasks. These findings establish a separation between using contextual skills and reusing their internal states, and demonstrate cache-assisted recovery across models and tasks.

CCS Concepts

• **Computing methodologies** → **Natural language processing**. *Knowledge representation and reasoning*.

Keywords

agent skills, Web agents, large language models, in-context learning, activation patching, context reuse, mechanistic interpretability

1 Introduction

Language models can perform new tasks by following instructions in their context. A skill makes such instructions reusable: a calculator skill, for example, specifies a formula, input conventions and the steps for applying them to a new question. A model divides text into tokens and represents each position with an internal vector, or hidden state. If the hidden states formed from these instructions also support reuse, one execution could supply task-relevant information to another without changing the model's weights.

Agent Skills organize instructions, scripts and resources around a `SKILL.md` entry point [32]. Web agents use reusable skills and workflows to guide later actions [28, 29]. This motivates studying whether the internal states formed from such instructions can also be reused. We study this question through controlled question answering after a skill has been selected, isolating the use and transfer of its instructions.

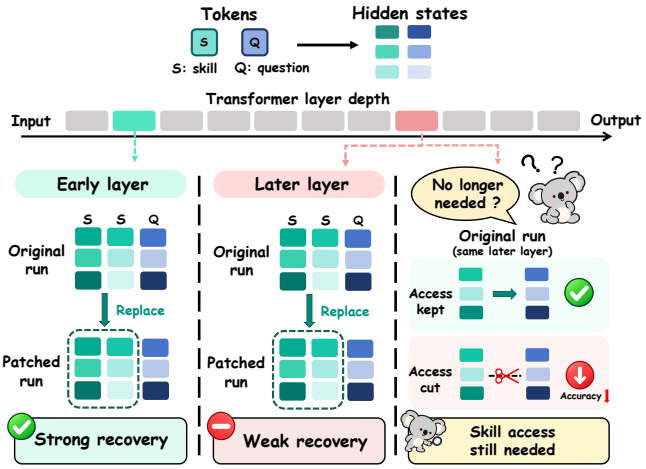


Figure 1: Weak transfer and continued skill use at the same later depth. Copying skill-position states from a correct-skill run into an unrelated-skill run gives strong early recovery (left) and weak late recovery (middle). At that same later depth, the original run still needs skill access: retaining access preserves performance, while cutting it reduces accuracy (right). S and Q label skill and question tokens. Figure 3 gives the measured depth profiles.

The difficulty is interpreting a failed transfer. If copying a skill's hidden states no longer restores performance, has the model stopped needing those instructions, or has the copy failed to supply what the receiving execution needs? Task vectors summarize task demonstrations, while function vectors combine selected internal attention outputs. Both show that internal states can transmit task behaviour across contexts [13, 24]. Yet restoring an execution and disrupting a successful one measure different effects [12]. This motivates our central question: *can skill-state transfer weaken while the original execution still relies on access to the skill?*

We answer this question with two tests across model depth. The first copies hidden states at the skill's token positions from a correct-skill execution into an execution using length-matched text from an unrelated skill. The second blocks attention to the skill in the original correct-skill execution, from a chosen layer onward. Content, position and cross-question controls first establish that early copying transfers skill-relevant information. We then find a consistent separation in eight evaluated combinations of four models and two benchmarks: transfer weakens at later layers while the original execution remains strongly dependent on skill attention (Figure 1).

WWW '27, Dublin, Ireland
2027.

We next test whether adding information omitted by hidden-state copying can improve late transfer. Attention stores information from previously processed tokens in a key/value (KV) cache. At layer 16, copying these caches together with hidden states improves accuracy over state transfer alone in all eight settings, by 6.8 to 43.4 percentage points. Blocking subsequent skill reading reduces the recovered accuracy in each setting. For Qwen3-8B/MedCalc, joint replacement raises accuracy from 3.5% to 42.1%, and the reading cut reduces it to 7.0%. The shared direction across models and tasks establishes cache-assisted partial restoration alongside continued use of the skill.

Additional complete-source controls show that copying every layer of skill-position KV is still insufficient to reproduce the donor. Copying the complete prefix KV reproduces donor outputs, while the early-versus-late joint-repair pattern varies across models and tasks.

The paper follows three steps: validate early transfer (§4.2), establish its separation from continued skill use (§4.3), and test the extent of cache-assisted restoration (§4.4). Together, the tests distinguish the ability to use contextual instructions from the ability to reuse their internal states, and identify cached information as a component that improves reuse.

2 Related Work

State transfer and its interpretation. Task vectors summarize demonstrations as internal vectors [13], and function vectors identify attention outputs that transmit task behaviour [24]. Instructions also yield transferable components [2, 6], whose effects vary with intervention depth [18]. Activation patching, which copies internal states between executions, requires matched controls and careful implementation [12, 33]. In particular, restoring behaviour and disrupting it need not reveal the same components. Interactions with other components can also affect a patching estimate [26]. We use these methodological distinctions to compare skill-state transfer and source dependence across depth.

Depth, source access and context caches. Pola and Balasubramanian [20] locate an onset beyond which selected hidden-state replacements largely cease to change answers in their instruction tasks. Sia et al. [22] use layer-wise context masking, which blocks attention to selected context positions, in machine translation and code generation to locate where attention to the instructional context becomes unnecessary. Joint transfer and masking analyses also locate subtask computation [31]. We instead define the source as an explicit skill-instruction span and compare transfer into a wrong-skill execution with blocking attention to that span in the original execution on common questions. The comparison measures how the two effects separate across depth.

PrefillNotes [16] studies editable and reusable KV caches. In its field-editing tasks, replacing local KV entries can fail when downstream cached conclusions still reflect the old field. It also composes precompiled skill caches into new contexts. Our focus is the depth separation between skill-state transfer and original-run source dependence. We then ask whether copying source caches together with states partially restores late transfer.

Skills and reusable context. Agent Skills organize instructions and resources for on-demand use [32]. Benchmarks study skill retrieval and application [11, 17, 23], while matched-budget evaluations expose their dependence on available computation [10]. We hold skill selection fixed. Other approaches compress context into aggregated activations [1], learned summary tokens [19], hidden states [8], or weight updates [3, 4]. Our interventions characterize naturally occurring states and source dependence without training a compression mechanism.

3 Problem Formulation

Skills, executions and token positions. We supply a skill's instructions directly and hold skill selection fixed. For the same question, the *donor* execution uses the correct skill and the *receiver* execution uses length-matched text from a fixed unrelated skill (Appendix A.1). The skill's token positions, denoted $\mathcal{I}$, form the *source span*. For example, a calculator's formula and input conventions belong to this span. The question does not. *Readers*, denoted $\mathcal{J}$, are positions after the source span, including subsequent prompt tokens and generated answer tokens. Thus receiver names an execution, whereas reader names a token position that can attend to the skill. Causal attention prevents a position from reading future tokens, so tokens before the source cannot read it.

Layers are indexed from zero. The model processes prompt tokens during *prefill* and generates answer tokens during *decoding*. We use greedy decoding, which selects the highest-probability next token. Our main comparison asks what transferring source states restores in the receiver and what blocking source attention removes from the donor.

Hidden-state transfer. Each Transformer block is a processing layer that updates the token vectors. Let H^L denote its output at layer L, also called residual states. Subscripts D and R denote donor and receiver. State replacement, labelled S, writes

$$S: \qquad H_R^L[\mathcal{I}] \leftarrow H_D^L[\mathcal{I}]. \tag{1}$$

The write occurs during prefill, after which the receiver continues with the patched states. Other positions at that layer remain unchanged.

Blocking attention to the skill. In the original correct-skill execution, we block attention to source positions at a chosen layer L and every subsequent layer. We call this *cumulative source-attention knockout*. In attention, a query vector scores keys at available positions, and the resulting weights combine their value vectors. Let z_{ij}^{ℓ} be the attention score from query position i to key position j, including the causal mask that blocks future tokens. At each layer $\ell \geq L$, we set

$$\tilde{z}_{ij}^{\ell} = \begin{cases} -\infty, & j \in \mathcal{I}, \\ z_{ij}^{\ell}, & j \notin \mathcal{I}. \end{cases} \tag{2}$$

The softmax operation converts scores into weights that sum to one. It gives blocked positions zero weight and normalizes the remaining weights. An attention head is one of the block's parallel attention components. The cut applies to all query positions and attention heads during prefill and decoding. It blocks both readers' access to the skill and attention within the skill itself, so its effect measures aggregate source-attention dependence.

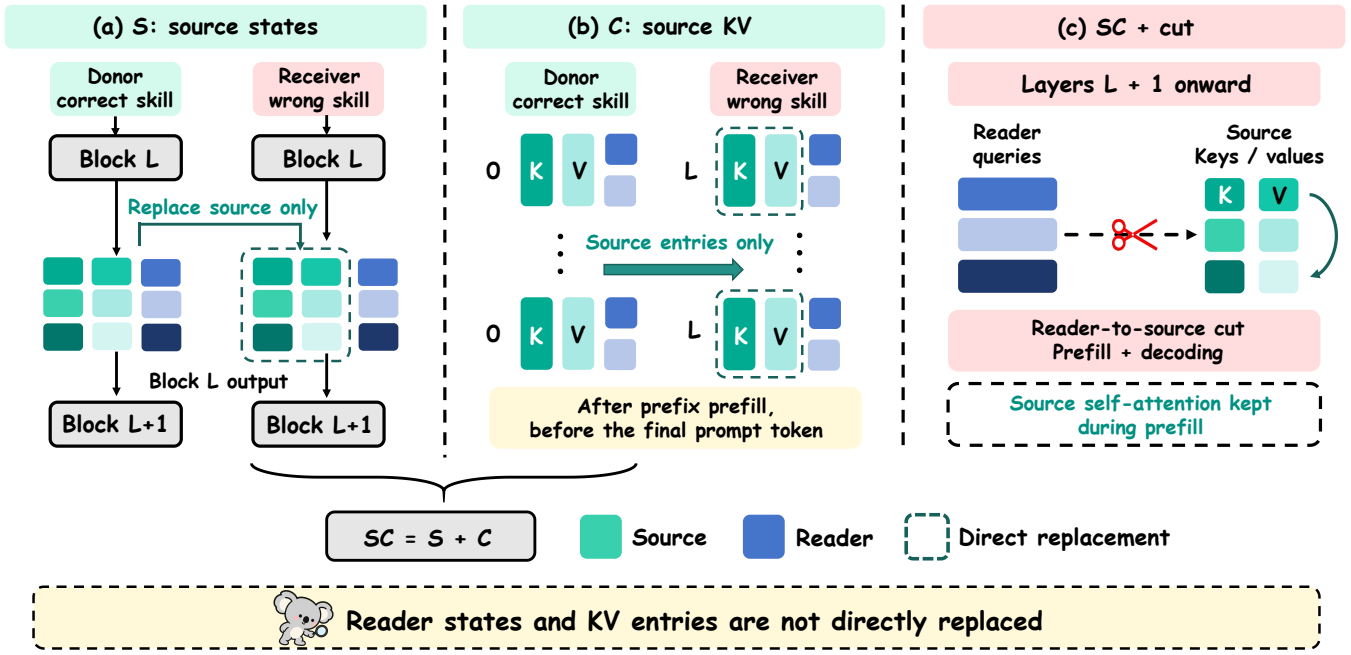


Figure 2: Timing of state repair, cache repair and the reading cut in the receiver. The prefix contains all prompt tokens except the last. (a) S copies donor source states at block L's output during prefix prefill. (b) C copies source-position key/value (KV) entries at layers 0 through L after prefix prefill, before final-token recomputation. SC combines these two writes, then recomputes the final prompt token and generates. (c) SC + cut installs a reading mask before prefill, blocking reader-to-source attention from $L+1$ onward during prefill and decoding. Source-to-source attention remains unmasked. Reader states and caches evolve through computation rather than direct replacement. Figure 4(b) separately tests cuts in the original correct-skill run.

The repair extension adds source-cache replacement, denoted C, and combines it with state replacement as SC. Figure 2 previews these operations, whose layer coverage is defined below.

Recovery and dependence. Write a_D, a_R and a_I for answer accuracy under the donor, receiver and an intervention I. Recovery is the fraction of the donor's gain restored in the receiver:

$$\rho(I) = \frac{a_I - a_R}{a_D - a_R}. \tag{3}$$

We require $a_D > a_R$. Thus $\rho = 1$ restores that gain and $\rho > 1$ exceeds donor accuracy. For state replacement at layer L, we write $\rho(L)$. All three accuracies use the same items and the transfer protocol's own unpatched baselines. Section 4.1 defines the evaluation sets.

For knockout, let $q(L)$ be the fraction of eligible, originally correct answers that remain correct. Source-attention dependence is

$$d(L) = 1 - q(L). \tag{4}$$

Higher d means more correct answers are lost when source attention is blocked. We compare ρ and d on common eligible question IDs, while retaining each test's baselines. The two measures describe restoration in the receiver and dependence in the donor. A separate paired-outcome check tests whether individual questions fail both interventions.

Depth boundaries. After establishing effective early transfer, we summarize the curves by their first half-effect boundaries:

$$\begin{aligned} T &= \min\{L : \rho(L) < 0.5\}, \\ R &= \min\{L : d(L) \le 0.5\}, \qquad \Delta = R - T. \end{aligned} \tag{5}$$

We report these boundaries when both crossings exist. The separation set $\mathcal{W} = \{L : \rho(L) < 0.5,\ d(L) > 0.5\}$ contains layers with weak transfer and strong source dependence. This set may contain several intervals. The thresholds summarize half-effect sizes, and Figure 3 retains later threshold crossings. Depth indexes processing layers within each token's computation.

The execution state before generation. For one question, split the complete n-token prompt before its last token u_*. The processed prefix has positions $\mathcal{P} = \{0, \ldots, n-2\}$, including the skill and question. Let $\overline{\mathcal{I}} = \mathcal{P} \setminus \mathcal{I}$. Attention reuses cached keys and values for these processed tokens [21]. At this point, define $C_X^\ell[\mathcal{I}] = (K_X^\ell[\mathcal{I}], V_X^\ell[\mathcal{I}])$, collect all layers as $C_X[\mathcal{I}]$, and write $\mathcal{B}_X = C_X[\overline{\mathcal{I}}]$ for the other prefix caches. With model parameters θ, positions, stopping rule and greedy decoding fixed, the continuation is

$$\hat{y}_X = \mathcal{G}_{\theta, M_X}(u_*, C_X[\mathcal{I}], \mathcal{B}_X). \tag{6}$$

Here M_X specifies attention masks, and $\mathcal{G}$ includes computing u_* and generating the answer from both source and other prefix caches.

Cache replacement and joint repair. After prefix prefill and before computing u_*, cache replacement writes

$$C: \quad C_X^\ell[\mathcal{I}] \leftarrow C_D^\ell[\mathcal{I}], \qquad 0 \le \ell \le L. \tag{7}$$

The prefix is computed without state replacement for C ($X = R$), or with the block-L write for SC ($X = S$). The post-prefill write preserves the other prefix caches. Thus $\mathcal{B}_C = \mathcal{B}_R$ and $\mathcal{B}_{SC} = \mathcal{B}_S$, although $\mathcal{B}_S$ can differ from $\mathcal{B}_R$ through indirect effects of S. Under the causal assumptions in Appendix B.2, ideal state replacement reconstructs source KV at layers above L. The explicit cache write supplies layers 0 through L, giving donor-aligned source caches under SC while retaining the receiver's other prefix caches $\mathcal{B}_S$. Figure 2 shows the write schedule.

From cache repair to output recovery. At this same observation point, flatten the prefix caches into $\mathbf{z}_X = (\mathbf{c}_X, \mathbf{b}_X)$, where $\mathbf{c}_X$ contains exactly the source KV entries at layers $0, \dots, L$. The remaining vector $\mathbf{b}_X$ includes higher-layer source KV as well as $\mathcal{B}_X$. Adding the cache write to S gives $\mathbf{z}_{SC} = (\mathbf{c}_D, \mathbf{b}_S)$ without changing $\mathbf{b}_S$. For $\varepsilon_X = \|\mathbf{z}_X - \mathbf{z}_D\|_2$,

$$\begin{aligned} \varepsilon_S^2 &= \|\mathbf{c}_S - \mathbf{c}_D\|_2^2 + \|\mathbf{b}_S - \mathbf{b}_D\|_2^2, \\ \varepsilon_{SC}^2 &= \|\mathbf{b}_S - \mathbf{b}_D\|_2^2, \\ \varepsilon_S^2 - \varepsilon_{SC}^2 &= \|\mathbf{c}_S - \mathbf{c}_D\|_2^2 \ge 0. \end{aligned} \tag{8}$$

Here $\|\cdot\|_2$ is the Euclidean length of a vector. For an exact copy, the coordinate decomposition proves that the squared cache distance decreases by precisely the replaced entries' squared error, independently of the causal reconstruction claim above. Appendix B.3 gives a sufficient condition for donor decoding: at every step, the change in each unnormalized token score, called a logit, must be less than half the donor's margin. The margin is the difference between its highest and second-highest scores. The condition includes cache updates and stopping.

Reading cuts and their timing. A *skill-reading cut* blocks reader-to-source attention. A *within-skill attention cut* blocks source-to-source attention. In SC + cut, the reading mask is

$$\widetilde{z}_{ij}^\ell = \begin{cases} -\infty, & \ell \ge L+1,\ i \in \mathcal{J},\ j \in \mathcal{I}, \\ z_{ij}^\ell, & \text{otherwise.} \end{cases} \tag{9}$$

It is installed before prefix prefill and persists during decoding. Source-to-source attention remains unmasked, but reader computation can change already during prefill. Consequently both M_X and $\mathcal{B}_X$ in Eq. (6) can differ from SC. The separate original-run path tests start their cuts at L, not $L+1$. Adding a within-skill cut to a persistent reading cut is structurally redundant for the output (Appendix B.1).

Three repair contrasts. On one fixed evaluation set under one protocol, let $a_X(L)$ denote accuracy for condition X. Suppressing L on the right, define

$$\begin{aligned} \Delta_{\text{cache}}(L) &= a_{SC} - a_S, \\ \Gamma(L) &= (a_{SC} - a_S) - (a_C - a_R), \\ \Delta_{\text{read}}(L) &= a_{SC} - a_{SC+\text{cut}}. \end{aligned} \tag{10}$$

These measure the added cache benefit, the interaction on the accuracy scale, and sensitivity of joint repair to a reading cut. Positive Γ means the joint gain exceeds the sum of the individual gains. These are intervention contrasts on the accuracy scale. Multiplying them by 100 gives percentage points.

4 Experiments

The experiments follow three questions. Does early state transfer carry skill-relevant information? Does that transfer weaken before dependence on the skill does? Can source-cache replacement restore part of the late transfer deficit?

4.1 Experimental setup

Tasks and models. We pair MedCalc-Bench [15] with skills from SRA-Bench, a benchmark that pairs questions with reusable skill documents [23], covering 1,100 questions from 55 clinical calculators. We also use SRA-Bench's TheoremQA subset, with 747 questions and 320 skills. TheoremQA tests the application of theorems across mathematics, physics, electrical engineering and computer science, and finance [5], extending the evaluation beyond clinical calculations. Each question receives its designated skill. Skill selection is held fixed.

The four main models are Qwen3-8B [30], Mistral-7B-Instruct-v0.3 [14], Llama-3.1-8B-Instruct [9] and DeepSeek-LLM-7B-Chat [7]. The depth study covers all eight combinations of model and benchmark (Table 2). Qwen3-8B and Mistral on TheoremQA use regenerated answers with independently checked grading (Appendix A.2). Models generate step-by-step answers without tools using greedy decoding, and the benchmark scorers evaluate those answers. On the full MedCalc benchmark, Qwen3-8B achieves 74.2% accuracy with the correct skill, 34.5% without one, and 31.5% with an unrelated skill.

Analysis populations. We select questions using earlier with-skill and without-skill outcomes, then re-evaluate unmodified runs, or baselines, under each protocol. The group receiver screen retains a skill only if its receiver solves none of its sampled questions.

For Qwen3-8B/MedCalc, we use 135 questions for controls (Table 1), 134 for common depth curves (Figure 3), 124 for the paired-outcome check (§4.3), and 114 for repair and path cuts (Figure 4). The control donor answers 124/135 correctly. The common depth set intersects transfer eligibility with correct-with-skill, incorrect-without-skill knockout outcomes. The paired check also requires both donors to be correct.

The added control cells require correct donor and incorrect no-skill baselines. Repair uses that rule plus an incorrect receiver for each item. Llama and DeepSeek reuse fixed candidate pools selected by earlier models. Intervention estimates are conditional on these selection rules. Each depth panel uses common question IDs for its two interventions. Repair and path tests likewise share question IDs within each model and task combination, using their own fresh-baseline screen. Appendix A details the distinct selection rules.

Protocol consistency. Historical Qwen3-8B and regenerated Qwen/Mistral TheoremQA runs use different attention implementations for transfer and knockout. Llama and DeepSeek use matching implementations. Repair and path cuts share a protocol with fresh baselines. Identity controls copy receiver states back to their original positions and reproduce its output tokens exactly. Resetting the TheoremQA evaluator before every answer and

Table 1: State-transfer controls at layer 8 across four models and two benchmarks. Entries are accuracy (%). n/G counts questions and skills. Qwen3-8B/MedCalc retains re-evaluated donor failures. The other cells additionally require a correct donor and incorrect no-skill baseline. The two matched-prefix rows replace the same number of leading skill positions with correct or unrelated content. Final-token controls replace fewer positions. Layer 8 is already beyond the transfer boundary for some models (Table 2).

Intervention	Qwen3-8B		Mistral-7B		Llama-3.1-8B		DeepSeek-7B	
	MedCalc	TheoremQA	MedCalc	TheoremQA	MedCalc	TheoremQA	MedCalc	TheoremQA
n/G	135/14	57/49	35/13	61/51	112/20	92/71	25/13	54/45
Correct skill donor	91.9	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Wrong skill receiver	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Correct skill span	**81.5**	**78.9**	**20.0**	**37.7**	**33.0**	**23.9**	**8.0**	**11.1**
Last prompt token	3.0	10.5	5.7	1.6	2.7	12.0	0.0	1.9
Last 16 prompt tokens	5.9	12.3	8.6	6.6	2.7	4.3	4.0	11.1
Correct matched prefix	59.3	71.9	17.1	37.7	35.7	22.8	12.0	7.4
Other-skill matched prefix	4.4	14.0	2.9	8.2	8.0	9.8	8.0	5.6
Correct span, permuted	4.4	22.8	14.3	14.8	4.5	14.1	4.0	5.6
Norm-matched noise	2.2	19.3	8.6	4.9	4.5	9.8	4.0	0.0
Identity patch	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

reversing grading order preserve the labels. Implementation details and corrections are in Appendix A.

4.2 Validation of skill-state transfer

Early transfer recovery. The depth comparison requires a state-transfer test that works at early layers. At layer 8 on the 135-item Qwen3-8B/MedCalc set, copying states at the skill positions yields 110 correct answers. The donor answers 124 of the same 135 questions correctly and the receiver answers none, giving aggregate recovery $\rho = 110/124 = 0.89$ (Table 1). Patching the last prompt position or the final 16 positions has much smaller effects. The skill span is therefore an effective intervention site. The matched-prefix controls below compare content at equal replacement lengths.

Content and position controls. The paired content controls replace a common prefix length with either correct-skill or other-skill information. Correct content recovers 0.65 of the donor's gain, versus 0.05 for other-skill content. Shuffling source positions and injecting random vectors of matching length also give little recovery. The latter is the norm-matched noise control, which preserves the Euclidean length of the skill-state change. Rescaling the correct patch to the receiver's norm still recovers 0.87. Together, these controls support a role for skill-specific content and its arrangement. The expanded table reports the same controls for every model and task. Since layer 8 follows the transfer boundary in several cells, its lower recovery is consistent with Figure 3. An additional layer-2 check gives 56.0% to 93.4% span accuracy across the eight cells, above noise and shuffled-position controls in each case (Appendix A.2).

Cross-question transfer. In the main prompt order, the skill precedes the question. Causal attention prevents those skill states from encoding a later question. We additionally place the skill after the question and take donor states from other questions using the same skill. On 151 eligible items from 17 calculators, these states recover 0.98 at layer 8, close to 1.02 for a donor answering the current question. At layer 16, the values separate to 0.29 and 0.49. Early transfer thus carries information shared across questions, with greater question sensitivity later.

4.3 Weak transfer with persistent source dependence

Depth separation. Early transfer is effective, but it weakens before source dependence does. Figure 3 shows the full curves, and Table 2 summarizes their boundaries.

The dependence boundary follows the transfer boundary in all eight comparisons. The gaps are 8 to 20 layers, or 22.2% to 62.5% of model depth (Table 2). The common direction spans both benchmarks and all four model families.

A representative depth profile. For Qwen3-8B/MedCalc, recovery drops from 0.73 at layer 12 to 0.16 at 13 and stays below 0.08 from 15 onward. Dependence remains 0.93 at 13 and 0.75 at 24. Throughout layers 13 to 24, transfer restores less than half of the donor's gain while knockout removes more than half of the originally correct answers (Figure 3).

Paired outcomes and robustness. The separation also occurs within questions. In the Qwen3-8B/MedCalc paired check, 100/124 (80.6%) fail both interventions at layer 13, and 113/124 (91.1%) at 16 (Appendix B). The boundary ordering also holds at thresholds 0.4 and 0.6 in all eight comparisons. Broader screening and later threshold crossings appear in Appendix B.

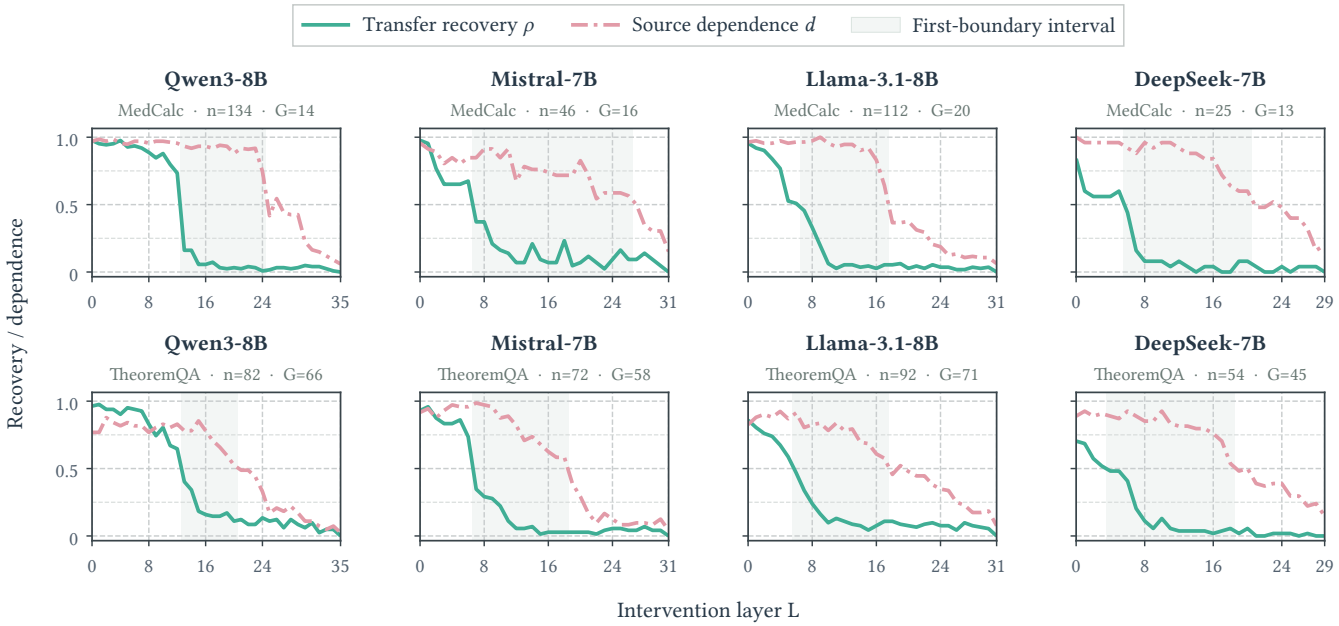


Figure 3: State transfer weakens before source-attention dependence across eight combinations of model and task. Rows show MedCalc (top) and TheoremQA (bottom), with models aligned by column. Mint: recovery $\rho(L)$ in the wrong-skill receiver. Pink dash-dotted: dependence $d(L)$ in the original correct-skill run. Each panel uses common eligible questions with each protocol's own baselines. G counts skills. Shading spans the first half-effect boundaries (Table 2). Later threshold crossings remain visible. Knockout blocks both skill reading and within-skill attention. Every layer is measured.

Table 2: Main depth comparison across four open-weight models and two benchmarks. n/G gives common eligible questions/skill groups. T and R are the first half-effect crossings for transfer ($\rho < 0.5$) and aggregate source dependence ($d \le 0.5$). $\Delta = R - T$ is in layers. Δ/D normalizes the gap by the model's layer count. All eight cells use audited common-item populations.

Model	MedCalc-Bench					TheoremQA				
	n/G	T	R	Δ	Δ/D (%)	n/G	T	R	Δ	Δ/D (%)
Qwen3-8B	134/14	13	25	12	33.3	82/66	13	21	8	22.2
Mistral-7B	46/16	7	27	20	62.5	72/58	7	19	12	37.5
Llama-3.1-8B	112/20	7	18	11	34.4	92/71	6	18	12	37.5
DeepSeek-7B	25/13	6	21	15	50.0	54/45	4	19	15	50.0

4.4 Partial restoration with source-cache replacement

Testing an omitted source component. Figure 4 compares all eight model and task combinations, separating receiver repair at $L16$ (panel a) from attention cuts in the original correct-skill run at $L8$, $L16$ and $L24$ (panel b).

Repair gains in the receiver. Adding caches improves accuracy over state transfer alone in all eight layer-16 comparisons, by 6.8 to 43.4 percentage points (Figure 4a). Joint-repair accuracy ranges from 10.2% to 62.3%. For Qwen3-8B/MedCalc, S, C and SC answer 4, 8 and 48 of 114 questions correctly (3.5%, 7.0%, 42.1%). The cache benefit is $\Delta_{\text{cache}} = (48 - 4)/114$, or 38.6 percentage points. The accuracy interaction is $\Gamma = (48 - 4 - 8 + 0)/114$, or 31.6 points (Eq. (10)). At layer 24, SC reaches 25.4%, versus 0.9% for S and 18.4% for C. The added benefit of states alongside caches varies. For Mistral/MedCalc at $L16$, SC and C both reach 18.4%, with $\Gamma = -7.9$ points. Table 4 reports all individual conditions and contrasts.

Complete source KV and receiving history. We extend the complete-source controls to all eight model/task combinations, using fixed historical pools totaling 580 model–question pairs. Qwen3-8B/MedCalc reuses the 124-item experiment; the other seven cells add 456 items. Each cell uses matched BF16, eager attention, split prefill and greedy decoding in unrelated-skill and fixed unrelated-text receivers (Wrong/Neutral). These pools retain fresh baseline failures and differ from the repair screen above. Across all eight cells, every source-position K/V tensor at every layer matches

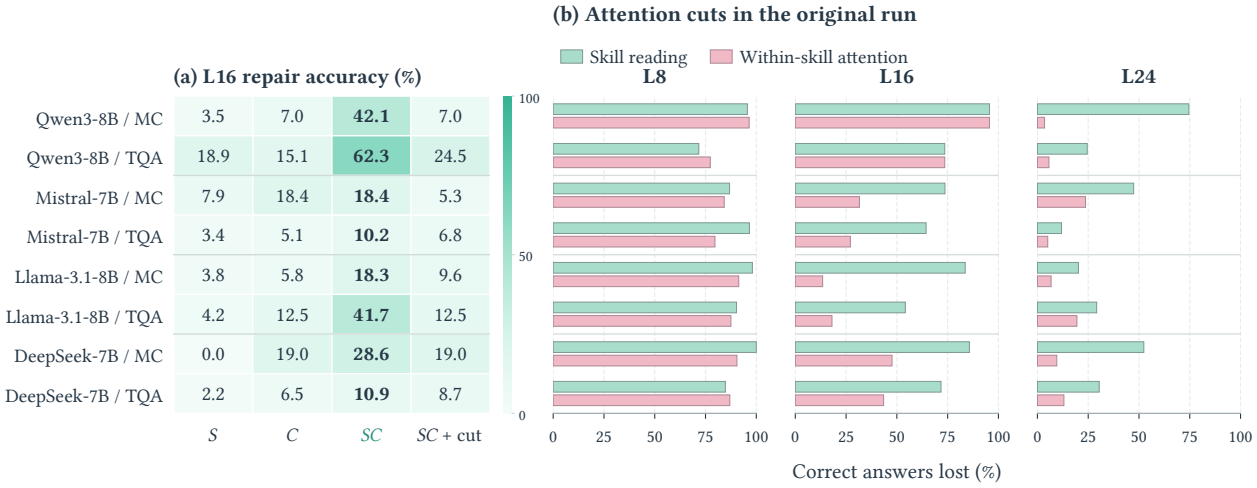


Figure 4: Repair and attention cuts across four models and two benchmarks. MC denotes MedCalc and TQA denotes TheoremQA. (a) Receiver accuracy at $L16$ for state replacement (S), cache replacement (C), joint repair (SC), and joint repair with a reading cut from $L17$ onward (SC + cut). (b) Correct answers lost in the original correct-skill run under cuts from $L8$, $L16$ or $L24$ onward during prefill and decoding. Skill reading blocks reader-to-source attention. Within-skill attention blocks source-to-source attention. Each row uses the same questions in both panels, with counts in Table 4. Layer numbers index each model's blocks.

the donor under SC at $L8$, $L16$ and $L24$ before continuation. An additional explicit all-layer write changes none of the 3,480 paired generated sequences. Nevertheless, copying only all source KV reaches 9.3–58.7% in Wrong and 17.4–73.0% in Neutral, below each cell's donor (85.2–97.6%; Table 3).

Depth effects depend on the receiving execution. With the same complete source KV, SC@8 exceeds SC@24 in both contexts in six of eight cells. The direction is mixed for Llama/TheoremQA (−5.4/+6.5 points in Wrong/Neutral) and DeepSeek/MedCalc (+8.3/−8.3); all four corresponding skill-group intervals include zero. Five cells have positive exploratory 95% intervals in both contexts; Qwen/TheoremQA's Neutral interval touches zero (Appendix B.4). Thus the large Qwen3-8B/MedCalc depth gap is not a uniform cross-model rule.

Copying the complete prefix KV instead reproduces donor tokens in all 1,160 receiver–question pairs. Copying all source KV and recomputing the later prompt reaches within 4.2 accuracy points of the donor in every cell, although it is not a token-level identity. Together, these controls identify receiving state outside the skill positions as a factor in recovery after source KV is complete. They establish neither a minimal state to copy nor a low-cost reuse method. Content and layer-window controls remain a Qwen3-8B/MedCalc case study (Appendix B.4).

Reading under joint repair. Reading cuts reduce joint-repair accuracy by 2.2 to 37.7 points across all eight layer-16 comparisons. For Qwen3-8B/MedCalc, accuracy falls from 42.1% to 7.0%, giving $\Delta_{\text{read}} = (48 - 8)/114$, or 35.1 points. Thus the cross-model improvement from adding caches coexists with continued use of the skill. The Qwen3-8B/MedCalc result at $L24$ differs: the cut gives 27.2% versus 25.4% without it. Appendix B.1 documents the layer grid and checks sensitivity to outputs stopped at the generation limit.

Attention paths in the original run. Figure 4(b) tests the original correct-skill execution. At $L8$, both types of cut cause large losses across the eight rows. At $L24$, Skill reading loses 11.9% to 74.6% of correct answers, exceeding Within-skill attention by 6.8 to 71.1 points in every row. The Qwen3-8B/MedCalc contrast is 74.6% versus 3.5%. At this measured depth, later positions' access to the skill has a larger effect than attention among skill positions across all eight settings.

5 Discussion

Potential applications. Our findings suggest a way to evaluate the reuse of contextual skills. Consider a Web agent that repeatedly follows the same instructions across requests. A reusable internal representation would need to preserve task performance in the receiving execution. Our results motivate evaluating this transfer separately from the original execution's dependence on the skill. If a reuse method also removes access to the original instructions, that additional change needs its own evaluation. The observed cache gains and receiving-history effects motivate reuse methods that account for both source representations and the receiving execution's cached state. Deployment studies can evaluate their latency, memory use and task performance in interactive Web settings.

Future work. A next step is to investigate which questions benefit from cache replacement. The mathematical analysis connects remaining cache differences to output stability under explicit assumptions. Measuring these differences, their influence on token predictions and the donor's prediction margins could test whether this framework helps distinguish successful from unsuccessful repairs. The complete-source controls further motivate testing which parts of the receiving history must be recomputed, and whether the required budget can be predicted on held-out skills.

Table 3: Complete-source controls across four models and two benchmarks. Entries are accuracy (%). MC and TQA denote MedCalc and TheoremQA. Wrong and Neutral are the receiving contexts; Base is their unmodified accuracy. C_{all} copies source KV at every layer; *SC* additionally patches source hidden states at the stated layer. *B* copies only other prefix KV, Full copies the complete prefix KV, and Replay copies all source KV then recomputes the later prompt. Full also gives donor accuracy. These pools differ from Table 4. Explicit all-layer rewriting after *SC* leaves every generated token unchanged. Bold marks the highest accuracy among the three *SC* depths in each row (ties included).

Model / task	Context	Base	C_{all}	*SC*@8	*SC*@16	*SC*@24	*B*	Full	Replay
Qwen3-8B / MC	Wrong	4.8	29.8	**91.9**	41.9	25.8	2.4	97.6	96.8
	Neutral	4.0	25.0	**91.1**	28.2	26.6	8.1	97.6	96.8
Qwen3-8B / TQA	Wrong	6.3	58.7	**95.2**	60.3	50.8	20.6	96.8	98.4
	Neutral	25.4	73.0	**84.1**	65.1	71.4	34.9	96.8	98.4
Mistral-7B / MC	Wrong	6.5	15.2	**56.5**	15.2	13.0	4.3	91.3	89.1
	Neutral	6.5	17.4	**39.1**	17.4	17.4	19.6	91.3	89.1
Mistral-7B / TQA	Wrong	3.1	10.8	**49.2**	9.2	13.8	24.6	93.8	93.8
	Neutral	4.6	23.1	**43.1**	21.5	21.5	27.7	93.8	93.8
Llama-3.1-8B / MC	Wrong	1.8	16.1	**73.2**	17.0	16.1	1.8	96.4	96.4
	Neutral	8.0	27.7	**48.2**	25.9	25.9	4.5	96.4	96.4
Llama-3.1-8B / TQA	Wrong	5.4	43.5	45.7	46.7	**51.1**	10.9	90.2	89.1
	Neutral	10.9	46.7	**57.6**	**57.6**	51.1	16.3	90.2	89.1
DeepSeek-7B / MC	Wrong	0.0	29.2	**41.7**	29.2	33.3	20.8	91.7	87.5
	Neutral	8.3	41.7	25.0	29.2	**33.3**	20.8	91.7	87.5
DeepSeek-7B / TQA	Wrong	0.0	9.3	**22.2**	9.3	9.3	14.8	85.2	83.3
	Neutral	14.8	31.5	**44.4**	31.5	29.6	22.2	85.2	83.3

The heterogeneous depth effects suggest that a recomputation rule should be tested across model/task cells rather than inferred from one depth curve. Separating attention cuts during prompt processing from those during answer generation would clarify when continued skill access matters. Finally, multi-step Web workflows would assess how far the observed separation extends beyond the question-answering settings studied here.

6 Conclusion

Using a contextual skill and reusing its internal states are distinct capabilities. Across four models and two benchmarks, state transfer weakens before the original execution's dependence on skill attention, with half-effect boundaries separated by 8 to 20 layers. Early content controls establish that the transferred states carry skill-relevant information, and the depth comparisons show that this information becomes harder to reuse while the skill remains useful.

Source-cache replacement provides a way to recover part of that transfer. At layer 16, adding caches improves accuracy over states alone by 6.8 to 43.4 percentage points in all eight comparisons, while cutting subsequent skill reading reduces the joint-repair accuracy in each. Complete-source controls further show that skill-position KV alone is insufficient to recover donor performance. Full-prefix copying reproduces the donor, while the depth effect of joint repair varies across models and tasks. The mathematical analysis connects cache alignment to output stability under explicit conditions. Together, these findings motivate evaluating state reuse separately from continued skill use and including cached information in the design of reusable skill representations.

References

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. doi:10.18653/v1/2026.acl-long.559

[3] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[5] Wenhu Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. 2023. TheoremQA: A Theorem-driven Question Answering Dataset. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. https://arxiv.org/abs/2305.12524 arXiv:2305.12524.

[6] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models? *arXiv preprint arXiv:2505.12075* (2025). https://arxiv.org/abs/2505.12075

[7] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[8] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[10] Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *arXiv preprint arXiv:2606.15017* (2026). https://arxiv.org/abs/2606.15017

[11] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[12] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[13] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[15] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[16] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. *arXiv preprint arXiv:2606.17107* (2026). https://arxiv.org/abs/2606.17107

[17] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[18] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[19] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[20] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract. arXiv:2511.10694.

[21] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150* (2019). https://arxiv.org/abs/1911.02150

[22] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.52202/079017-1030

[23] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[24] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[25] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. 2018. Lipschitz-Margin Training: Scalable Certification of Perturbation Invariance for Deep Neural Networks. *arXiv preprint arXiv:1802.04034* (2018). https://arxiv.org/abs/1802.04034

[26] Sankaran Vaidyanathan, David Arbour, Aaron Mueller, Scott Niekum, and David Jensen. 2026. The Curse of Multiple Mediators: Hidden Interaction Effects in Activation Patching. *arXiv preprint arXiv:2606.27510* (2026). https://arxiv.org/abs/2606.27510

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. https://arxiv.org/abs/1706.03762

[28] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. WebXSkill: Skill Learning for Autonomous Web Agents. *arXiv preprint arXiv:2604.13318* (2026). https://arxiv.org/abs/2604.13318

[29] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[30] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[31] Zhipeng Yang, Junzhuo Li, Siyu Xia, and Xuming Hu. 2025. Internal Chain-of-Thought: Empirical Evidence for Layer-wise Subtask Scheduling in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 22536–22564. doi:10.18653/v1/2025.emnlp-main.1147

[32] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[33] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A Experimental protocols and provenance

Tasks and scoring. MedCalc provides 55 calculator skills and 20 questions per calculator through SRA-Bench, scored against prescribed numerical tolerances. TheoremQA uses its released prompts, skill assignments and evaluator. Skills are supplied directly, with no tools during generation. Groups are calculators for MedCalc and skill identities for TheoremQA.

Evaluation populations. The corrected Qwen3-8B/MedCalc transfer pool contains 467 of 470 historically rescued items from 48 calculators. Whole-group screening retains 135 items from 14 calculators, including donor failures: 124 donors and 110 layer-8 span patches are correct. Recovery uses this measured donor gain. The common transfer/knockout depth set has 134 items and separate intervention baselines. The seven added control cells require fresh donor success and no-skill failure. Repair instead screens individual items; Appendix B.1 gives its rule. Complete-source controls retain fixed candidates, including fresh errors (Appendix B.4). These populations are not interchangeable. The common DeepSeek/MedCalc depth set has only 25 items; threshold sensitivity does not establish robustness to sample composition.

A.1 Implementation details

Wrong-skill selection and token matching. Each dataset uses one fixed distractor. MedCalc uses APACHE II, a clinical severity score (`medcalcbench_043`), as a long distractor outside the selected calculators. TheoremQA uses `theoremqa_148`, a mathematical skill about representation theorems and differential equations, selected by fewest assigned questions, then greatest document length. Its assigned items are excluded. We repeat the distractor's tokens if needed and copy the first m into the target span of width m, without padding. All other tokens and positions remain unchanged. This context may repeat or truncate the alternative skill.

Prompt and source boundaries. The user turn has the order `Relevant Skill:`, skill text, a blank line, and the benchmark's question prompt. MedCalc appends the instruction to show calculations and end with `ANSWER: <your answer>`. Its system message asks for a step-by-step score calculation from a patient note. TheoremQA retains the benchmark's science-teacher system message, `Problem:<question>` and `Solution:` user format, and "Therefore, the answer is ..." final-answer requirement. The tokenizer converts text to tokens. Its chat template supplies role labels and the generation prompt. Character offsets identify tokens overlapping the skill text, excluding its header and separator. We replace IDs in $[\mathrm{lo}, \mathrm{hi})$, including the start but excluding the end, without decoding and retokenizing.

Models and decoding. The four main models are Qwen3-8B (36 layers, hidden width 4096), Mistral-7B-Instruct-v0.3 (32 layers, width 4096), Llama-3.1-8B-Instruct (32 layers) and DeepSeek-LLM-7B-Chat (30 layers). Transfer and knockout allow 900 new tokens, with greedy decoding and Qwen's extended reasoning disabled. Llama and DeepSeek use BF16 and eager attention; regenerated Qwen/Mistral TheoremQA uses SDPA for transfer and eager attention for knockout. DeepSeek and Mistral merge

system instructions into the user turn. Llama retains original weights/configuration/vocabulary with the public distribution's chat template. Repair and path experiments use BF16, eager attention and 1,536 new tokens. All their conditions split prefill before the final prompt token, allowing cache writes before its recomputation. All cuts span prefill and decoding. Layer indices in the path comparisons refer to the measured blocks, not matched computational stages across models.

Position and state checks. Length matching preserves question positions. Patches replace prefill block outputs indexed separately from scored answer tokens. Identity controls write back receiver states. A final-layer patch reproduces donor next-token scores at that position. Cuts preserve the causal mask.

Split-prefill check. On one item from each of 14 calculators, all 84 receiver-state checks reproduce tokens exactly. Splitting prefill changes one donor's and one receiver's correctness, so repair uses fresh matched baselines.

A.2 Evaluation scope and intervention provenance

Attention-mask correction. All main state interventions match capture and decoding attention masks. Earlier mask omission changed numerical implementations and failed identity checks; correction restores exact agreement, including all 467 corrected MedCalc items. Knockout requires no donor-state capture.

Extension pools and grading checks. Llama and DeepSeek are freshly screened on frozen pools of 446 MedCalc and 255 TheoremQA items inherited from earlier eligible transfer sets. One DeepSeek/MedCalc prompt cannot reserve 900 answer tokens and is excluded before inference. Qwen/Mistral TheoremQA uses 168/142 candidates, yielding 82/66 and 72/58 common questions/skills. Official grading resets parser state per answer; reverse-order and separate-process checks agree before recomputing eligibility.

Layer-2 controls use their own screened sets. In the model order of Table 1, the MedCalc question counts are 97, 35, 112, 25 and correct-span counts are 87, 24, 101, 14. TheoremQA counts are 61, 61, 92, 54, with 57, 52, 70, 31 correct. The DeepSeek/TheoremQA cells use the final 54-item grading correction.

Artifact availability. Stored artifacts contain configurations, question IDs, answers and token sequences. An anonymous review archive is not yet linked.

B Additional transfer and source-dependence analyses

The corrected Qwen3-8B/MedCalc rerun uses all 467 eligible questions from 48 calculators. Whole-group and individual-item receiver-failure screens retain 135/14 and 369/45 questions/calculators, respectively.

Broader screening and boundary sensitivity. The item-level screen retains the transfer decline. Paired knockout comparisons use the common-item populations. Using transfer thresholds $\tau = 0.4$ or 0.6 with dependence thresholds $1 - \tau$ preserves the boundary ordering in all eight main comparisons. Qwen3-8B/MedCalc dependence first falls to at most half at layer 25, rises above half again at 26, and remains below half from 27 onward. The curves retain these recrossings.

Same-question joint failures. This check requires correct donors under both protocols, an incorrect transfer receiver and an incorrect no-skill knockout baseline. Joint failures at the transfer boundary are 100/124 (80.6%) for Qwen3-8B at $T = 13$ and 25/43 (58.1%) for Mistral-7B at $T = 7$, on 14 and 15 MedCalc skill groups. These sets differ from Table 2. Four items in each pass the transfer no-skill baseline despite failing it under knockout.

B.1 Source-cache repair and attention-path experiments

Repair and path cuts use fresh donor-correct, receiver-incorrect and no-skill-incorrect items, shared within each model/task row of Table 4. For Qwen3-8B/MedCalc, this retains 114 of 124 frozen candidates from 14 calculators (116 pass the first two criteria). All 372 baseline generations agree token-for-token between suites, and all 48 receiver-state writebacks are token-exact. Repair uses $L8/L12/L16/L24$; path cuts use $L8/L16/L24$. Writes, masks and contrasts follow Eqs. (7), (9) and (10).

Qwen3-8B/MedCalc truncation sensitivity. All outcomes are retained, including generation-limit truncations. At $L16$, S, SC and SC + cut truncate 4, 4 and 3 of 114 outputs. The $SC - S$ gain is 38.6 points, 38.5 on the 96 items with untruncated baselines, and at least 35.1 under adversarial scoring of truncated S/SC answers. At $L24$, $SC/(SC + \text{cut})$ truncate 3/28 outputs without an additional measured accuracy loss.

At layer 24, reader and source-self cuts retain 29/114 and 110/114 correct answers. Their loss difference is 71.1 points. Reader cuts truncate 21 outputs. Counting all as correct still leaves retention at 50/114, below source-self retention 110/114 (no truncation). Both contrasts survive worst-case reassignment.

Redundant combined cut. With causal attention and otherwise positionwise operations, earlier tokens cannot read the skill. Persistent reading cuts also prevent its changed states from reaching later tokens. Adding a within-skill cut therefore leaves outputs unchanged by induction over layers and generated tokens. We omit this redundant condition. The argument does not apply when later reading is allowed.

B.2 Cache coverage under causal prefill

This deduction assumes a deterministic causal Transformer with masked attention as its only cross-position operation [27], identical parameters and position IDs, and the same exact-arithmetic computation without extra attention cuts. Donor and receiver tokens differ only within the complete, contiguous source interval $\mathcal{I} = [\text{lo}, \text{hi})$ in the prefix.

Proposition (source-cache coverage). Suppose S replaces the entire source output of block L before the next block executes, and the cache write occurs only after prefix prefill. At the observation point in Eq. (6),

$$C_S^\ell[\mathcal{I}] = \begin{cases} C_R^\ell[\mathcal{I}], & \ell \le L, \\ C_D^\ell[\mathcal{I}], & \ell > L. \end{cases} \tag{11}$$

Table 4: Source-cache repair at layer 16 across four models and two benchmarks. n/G counts questions and skills retained using fresh baselines. S, C, SC and SC + cut show accuracy (%). The last three columns show cache benefit Δ_{cache}, accuracy interaction Γ and reading-cut loss Δ_{read} in percentage points (Eq. (10)). Each row uses the same questions for all conditions, with donor accuracy 100% and receiver accuracy 0%. These populations differ from Table 2.

Model	Task	n/G	S	C	SC	SC + cut	Δ_{cache}	Γ	Δ_{read}
Qwen3-8B	MedCalc	114/14	3.5	7.0	**42.1**	7.0	+38.6	+31.6	+35.1
Qwen3-8B	TheoremQA	53/43	18.9	15.1	**62.3**	24.5	+43.4	+28.3	+37.7
Mistral-7B	MedCalc	38/15	7.9	18.4	**18.4**	5.3	+10.5	−7.9	+13.2
Mistral-7B	TheoremQA	59/49	3.4	5.1	**10.2**	6.8	+6.8	+1.7	+3.4
Llama-3.1-8B	MedCalc	104/20	3.8	5.8	**18.3**	9.6	+14.4	+8.7	+8.7
Llama-3.1-8B	TheoremQA	72/56	4.2	12.5	**41.7**	12.5	+37.5	+25.0	+29.2
DeepSeek-7B	MedCalc	21/10	0.0	19.0	**28.6**	19.0	+28.6	+9.5	+9.5
DeepSeek-7B	TheoremQA	46/40	2.2	6.5	**10.9**	8.7	+8.7	+2.2	+2.2

Table 5: Baselines and numerical identity checks for the complete-source study. Accuracy columns are percentages. n/G counts questions/skills. Replay equality counts token-identical sequences against the donor in each receiving context; the counts are the same for Wrong and Neutral. Complete-prefix copying is token-identical on all $2n$ receiver pairs per cell, and SC versus explicit all-layer rewriting is token-identical on all $6n$ pairs.

Model / task	n/G	Donor	No skill	Full copy	Replay tokens equal
Qwen3-8B / MC	124/14	97.6	3.2	97.6	48/124
Qwen3-8B / TQA	63/53	96.8	7.9	96.8	33/63
Mistral-7B / MC	46/16	91.3	2.2	91.3	28/46
Mistral-7B / TQA	65/55	93.8	0.0	93.8	53/65
Llama-3.1-8B / MC	112/20	96.4	1.8	96.4	55/112
Llama-3.1-8B / TQA	92/71	90.2	6.5	90.2	42/92
DeepSeek-7B / MC	24/12	91.7	4.2	91.7	12/24
DeepSeek-7B / TQA	54/45	85.2	0.0	85.2	34/54

Together with the explicit write at layers 0 through L, this gives $C_{SC}[\mathcal{I}] = C_D[\mathcal{I}]$.

Proof. Block-L KV is computed before the output patch, leaving source caches at $\ell \le L$ unchanged. Before lo, identical tokens and causality give identical states at every layer. After S, all block-L outputs at $i <$ hi therefore match the donor. At these positions, the next block reads only matching earlier or current states, producing matching source KV and outputs. Induction proves the upper-layer case. Equation (7) supplies the lower layers for SC. □

Remaining execution state. The post-prefill write preserves other-position caches: S and SC retain $\mathcal{B}_S$, while C retains $\mathcal{B}_R$. Thus,

$$\begin{aligned} \hat{y}_{SC} &= \mathcal{G}_{\theta,M_0}(u_*, C_D[\mathcal{I}], \mathcal{B}_S), \\ \hat{y}_D &= \mathcal{G}_{\theta,M_0}(u_*, C_D[\mathcal{I}], \mathcal{B}_D), \end{aligned} \tag{12}$$

where M_0 is the original mask. Since $\mathcal{B}_S$ need not equal $\mathcal{B}_D$, answers may differ. All-layer source-KV-only replacement instead preserves $\mathcal{B}_R$ and is generally not equivalent to SC.

Under SC + cut, source queries remain unmasked, so the same induction applies. However, $\mathcal{B}_{SC+\text{cut}}$ may differ from $\mathcal{B}_S$, and continuation uses a different mask. The contrast includes both prefill and decoding effects.

Code inspection confirms the stated write ordering. Runtime elementwise checks found zero source-cache discrepancies against the donor for SC at $L8$, $L16$ and $L24$ in both receiving contexts across all eight model/task cells (Appendix B.4). This verifies coverage for those BF16 runs; the proposition retains its exact-arithmetic assumptions.

B.3 A sufficient condition for behavioral recovery

For fixed model, positions, last prompt token and mask, let $F_t^D(\mathbf{z})$ map initial prefix caches to token-t logits after processing the donor history $y_{D,<t}$, including all intervening cache updates. Assume, over the compared states,

$$\|F_t^D(\mathbf{z}) - F_t^D(\mathbf{z}')\|_\infty \le \kappa_t \|\mathbf{z} - \mathbf{z}'\|_2. \tag{13}$$

Table 6: Paired depth effects with complete source KV. Entries give $SC@8 - SC@24$ in accuracy points and exploratory 95% skill-group bootstrap intervals. Positive values favor $L8$. Cap columns count outputs reaching the generation limit at $L8/L24$. Intervals are not adjusted for multiple comparisons.

Model / task	Wrong: gap [interval]	Neutral: gap [interval]	Wrong cap	Neutral cap
Qwen3-8B / MC	+66.1 [49.3, 78.5]	+64.5 [40.5, 82.8]	0/3	0/0
Qwen3-8B / TQA	+44.4 [32.2, 56.7]	+12.7 [0.0, 24.6]	0/0	0/1
Mistral-7B / MC	+43.5 [29.4, 61.1]	+21.7 [9.1, 35.3]	0/0	0/0
Mistral-7B / TQA	+35.4 [20.3, 50.0]	+21.5 [10.3, 33.3]	0/5	2/1
Llama-3.1-8B / MC	+57.1 [38.1, 72.5]	+22.3 [13.3, 29.5]	4/12	12/12
Llama-3.1-8B / TQA	−5.4 [−17.2, 7.2]	+6.5 [−4.8, 19.1]	8/3	4/5
DeepSeek-7B / MC	+8.3 [−6.9, 27.8]	−8.3 [−25.0, 16.7]	0/0	0/0
DeepSeek-7B / TQA	+13.0 [5.2, 22.6]	+14.8 [3.5, 26.9]	9/6	1/0

The infinity norm bounds the largest absolute logit change. Let $m_{D,t} > 0$ be the donor's highest logit minus its second-highest. Margin stability [25] gives

$$[\ \forall t \le N :\ 2\kappa_t \varepsilon_{SC} < m_{D,t}\ ] \quad \Longrightarrow \quad \hat{y}_{SC} = \hat{y}_D, \tag{14}$$

where N includes the stopping token and ε_{SC} is defined in Eq. (8).

Proof. Each logit changes by at most $\kappa_t \varepsilon_{SC}$, so the donor's winning logit remains ahead by at least $m_{D,t} - 2\kappa_t \varepsilon_{SC} > 0$. Induction over the shared greedy history, including the common stopping rule, proves sequence equality. □

This is only sufficient: different sequences may also answer correctly. Common sensitivity bounds give *SC* no larger error bound than *S*, but losing cancellation can increase actual logit errors, so accuracy need not improve. Sensitivities and itemwise margins remain unverified. *SC* + cut changes the mask and is excluded.

B.4 Complete-source and receiving-history controls

Population and protocol. The eight cells use 580 model–question pairs, retaining fresh baseline errors (Table 5). Qwen3-8B/MedCalc reuses all 124 frozen candidates from its 49-condition study. The other seven cells use pre-existing historical pools, capped at the first 124 IDs in their original order; one DeepSeek/MedCalc item fails input-length preflight, leaving 456 new pairs. Selection precedes repair outcomes. These are selected pools, not full-benchmark estimates or 580 independent questions across models; they differ from Table 4's screened populations. Wrong uses the original distractor. Neutral repeats or truncates one fixed unrelated document to match source positions; it may still influence answers. All conditions use BF16, eager attention, split prefill, greedy decoding and 1536 new tokens. TheoremQA resets parser globals per answer. Layers 8/16/24 are block indices, not matched fractions of model depth.

Conditions and numerical identities. The 24 conditions comprise four baselines (donor, Wrong, Neutral, no skill), *SC* and SC_{exact} at $L8/L16/L24$ in both receivers, and four further conditions per receiver: all-layer source-KV copying, other-position KV copying, complete-prefix KV copying, and all-layer source copying followed by replay of the later prompt. SC_{exact} explicitly rewrites every source K/V tensor after *SC*. Runtime elementwise checks find zero source discrepancies at all layers in every measured *SC* condition. The explicit rewrite preserves all 3,480 generated sequences across the eight cells. These identities concern KV at skill positions, not all residual states or all prefix KV. Other-position KV can still encode effects of earlier skill processing.

Complete-prefix copying reproduces all 1,160 donor sequences. Replay uses donor source KV and recomputes the later prompt; both require same-question donor computation. Replay is not exact: Qwen3-8B/TheoremQA scores 62/63 versus the donor's 61/63.

Paired uncertainty. Table 6 uses 10,000 whole-skill-group bootstrap resamples and percentile intervals (seed 20261024 for the seven new cells; Qwen3-8B/MedCalc reuses its audited analysis). Intervals are exploratory, unadjusted for multiple comparisons, and some cells contain few groups. Qwen3-8B/MedCalc's early-minus-late gap remains 63.7/64.5 points under adversarial scoring of capped outputs in Wrong/Neutral.

Additional Qwen3-8B/MedCalc content and layer controls. With all-layer source coverage fixed, correct-skill KV gives 29.8% in Wrong and 25.0% in Neutral. Unrelated-skill KV gives 4.8/6.5% and a second predeclared neutral text's KV gives 1.6/4.0%, respectively. Four equally sized eight-layer source writes, at layers 0–7, 8–15, 16–23 and 24–31, yield 3.2/3.2/1.6/4.0% in Wrong and 0.8/4.0/4.8/5.6% in Neutral. These windows individually recover little; comparison with all 36 layers does not identify the minimum required coverage because the write budgets differ.

Audit. Independent processes verify hashes, condition keys, cache writes, token identities and official scores in both orders. Exported labels reproduce all accuracies, cap counts and intervals for 10,944 new and 6076 reused formal records. Gate records are reused without regeneration.